



UNIVERSITÀ DEGLI STUDI DI SALERNO

Metodologie Utilizzate PT

Academy - Cybersecurity

Macchina Virtuale: no_name

Gruppo 11

Emiddio Ferrentino - 0612708848

Daniele Manzo - 0612709599

Andrea Rossomando - 0612708971

A.A. 2025/2026

Sommario

Il presente documento ha lo scopo di illustrare in maniera esaustiva e prettamente operativa le metodologie adottate durante l'attività di Penetration Testing effettuata sulla macchina virtuale denominata **no_name**. L'intero processo di valutazione ricalca le linee guida del Framework Generale per il Penetration Testing (FGPT). L'obiettivo primario è la documentazione accurata e riproducibile di ogni singola fase, evidenziando i vettori d'attacco scoperti, i comandi impartiti a terminale, gli script sviluppati e gli output significativi restituiti dai software di analisi.

Indice

1	Introduction	3
2	Phases FGPT	5
2.1	Target Scoping	5
2.2	Information Gathering	5
2.3	Target Discovery	6
2.4	Target Enumeration	7
2.4.1	OS Fingerprinting (Passivo e Attivo)	7
2.4.2	Fingerprinting Passivo (p0f & cURL)	7
2.4.3	Fingerprinting Attivo (Nmap)	8
2.4.4	Services Enumeration	9
2.4.5	Web Directory Enumeration (Gobuster & Owasp Dirbuster)	9
2.5	Vulnerability Assessment	12
2.6	Target Exploitation	12
2.7	Privilege Escalation	12
2.8	Maintaining Access	12
3	Final Consideration	13
4	References	14

Capitolo 1

Introduction

Il documento ha lo scopo di illustrare quelle che sono le metodologie adottate durante l'attività di Penetration Testing effettuata sulla macchina virtuale denominata **no_name**, distribuita da HacLabs, seguendo le fasi principali del FGPT. L'obiettivo principale è valutare il livello di sicurezza dell'infrastruttura attraverso un processo sistematico e riproducibile, documentando ogni fase con i relativi strumenti, comandi ed output significativi.

La macchina **no_name** è classificata come *beginner/intermediate* e presenta tre flag nascoste nelle directory home degli utenti. L'accesso iniziale è deliberatamente reso complesso, mentre la privilege escalation risulta più diretta una volta ottenuta una shell.

L'attività è stata condotta in un ambiente virtualizzato isolato, privo di connettività verso reti di produzione. Sono stati rispettati i seguenti vincoli operativi:

- Scope limitato alla macchina **no_name** (192.168.238.7)
- Nessuna azione distruttiva o alterazione persistente dei sistemi
- Utilizzo esclusivo di tool open-source e framework standard
- Documentazione completa di ogni azione eseguita

L'ambiente è stato configurato su VirtualBox con networking in modalità Host-Only, consentendo la visibilità reciproca tra le macchine nella subnet 192.168.238.0/24. Non vengono incluse ulteriori informazioni configurative in quanto non rilevanti ai fini del documento metodologico.

Di seguito sono elencati tutti gli strumenti impiegati durante l'attività, suddivisi per fase di utilizzo.

Tool	Versione / Note	Finalità
Netdiscover	v0.3.4	ARP Scanning - Target Discovery
Nmap	v7.99	Port scanning, OS fingerprinting attivo, service detection
Gobuster	v3.8.2	Directory brute-forcing su web server
OWASP DirBuster	v1.0-RC1	Directory brute-forcing (Interfaccia Grafica)
Nessus Essentials	Versione web	Vulnerability scanning automatizzato
OpenVAS	Versione web	Vulnerability scanning - report PDF
OWASP ZAP	Versione web	Web application scanning
Nikto	Versione CLI	Web server vulnerability scanner
Netcat (nc)	Traditional	Reverse shell listener / gestione connessione
Browser / DevTools	Firefox	Ispezione manuale del codice sorgente delle pagine web

Tabella 1.1: Classificazione e finalità dello stack tecnologico utilizzato.

Capitolo 2

Phases FGPT

2.1 Target Scoping

La fase di *Target Scoping* definisce in maniera rigorosa i confini dell'attività di Penetration Testing (le *Rules of Engagement*) e i vincoli operativi a cui il team di analisi deve attenersi.

Trattandosi di un ingaggio condotto su un ambiente di laboratorio di tipo CTF e non su un'infrastruttura di produzione, il perimetro autorizzato è stato concordato preventivamente e circoscritto in modo restrittivo alla singola macchina bersaglio. Qualsiasi asset o indirizzo IP non esplicitamente autorizzato è da considerarsi tassativamente fuori dal perimetro di test.

Di seguito si formalizzano i limiti logici ed etici dell'ingaggio:

In-Scope (Bersagli Autorizzati)

L'unico nodo su cui è consentito operare attività offensive, sia passive che attive, è l'host **no_name** (identificato nel capitolo successivo all'indirizzo IP 192.168.238.7). Rientrano nel perimetro autorizzato l'enumerazione di tutti i servizi di rete esposti (su protocolli TCP e UDP), l'attacco alle applicazioni web ospitate e le tecniche di *Privilege Escalation* a livello di sistema operativo locale.

Out-of-Scope (Bersagli Esclusi)

Sono categoricamente esclusi dai test di penetrazione il gateway dell'infrastruttura virtuale (192.168.238.2), l'host di servizio VirtualBox (192.168.238.6), il nodo attaccante Kali Linux (192.168.238.4) e qualsiasi altro segmento di rete estraneo al target primario.

Metodologie non consentite

Al fine di non compromettere l'integrità o la stabilità dell'ambiente di laboratorio, non è in alcun modo autorizzato l'utilizzo di tecniche che comportino la saturazione delle risorse o l'interruzione dei servizi, come attacchi di tipo *Denial of Service* (DoS / DDoS). Allo stesso modo, sono escluse dal mandato tutte le metodologie basate sull'*Ingegneria Sociale*.

2.2 Information Gathering

Prima di interagire attivamente con l'ambiente di laboratorio, la primissima fase di *Information Gathering* si è concentrata sull'analisi passiva della documentazione ufficiale fornita dal creatore dell'infrastruttura ([Haclabs](#)) sulla piattaforma di distribuzione VulnHub. Questa ricognizione informativa, assimilabile a un'attività di OSINT (Open-Source Intelligence), ha permesso di delineare le regole d'ingaggio e le peculiarità tecniche del bersaglio.

Dalla scheda tecnica dell'asset sono emersi i seguenti dati cruciali per l'orientamento delle operazioni:

- **Obiettivo dell'Ingaggio:** L'ambiente è strutturato secondo la metodologia *Boot2Root*. L'attaccante deve compromettere il sistema fino a ottenere i massimi privilegi amministrativi (*root*) e individuare

esattamente **3 flag** di verifica, posizionate dall'autore all'interno delle directory **home** degli utenti di sistema.

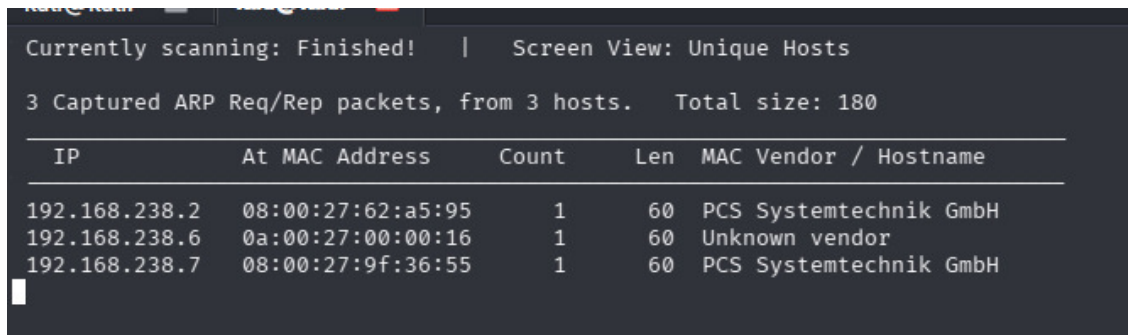
- **Profilazione della Difficoltà:** L'asset è classificato con un grado di difficoltà *Beginner/Intermediate*.
- **Vettori di Attacco (Hint dell'autore):** Le note di rilascio indicano esplicitamente che l'ottenimento dell'accesso iniziale (la prima *reverse shell*) richiederà uno sforzo analitico maggiore e risulterà "complicato", mentre le successive vulnerabilità per la *Privilege Escalation* dovrebbero rivelarsi più dirette. Questo dettaglio suggerisce al team di concentrare il massimo sforzo temporale nell'enumerazione iniziale dei servizi esposti.
- **Configurazione di Rete:** La macchina è preconfigurata per acquisire automaticamente un indirizzo IP tramite protocollo DHCP, confermando la necessità di procedere con uno *sweep* attivo della rete (Target Discovery) per individuarne l'esatta collocazione logica.

2.3 Target Discovery

La prima fase operativa ha previsto l'identificazione degli host attivi all'interno del segmento di rete locale. Operando in un ambiente *Black-Box*, è stato utilizzato il tool **netdiscover** per inviare richieste ARP (Address Resolution Protocol) in *broadcast* e mappare gli indirizzi IP associati ai vari MAC address presenti nel laboratorio.

Ricognizione di rete con Netdiscover

```
sudo netdiscover -r 192.168.238.0/24
```



The screenshot shows the output of the netdiscover command in a terminal window. The output indicates that the scanning process is finished and displays a table of captured ARP packets. The table has columns for IP, MAC Address, Count, Length, and Vendor/Hostname. Three hosts are listed: 192.168.238.2 (PCS Systemtechnik GmbH), 192.168.238.6 (Unknown vendor), and 192.168.238.7 (PCS Systemtechnik GmbH).

IP	At MAC Address	Count	Len	MAC Vendor / Hostname
192.168.238.2	08:00:27:62:a5:95	1	60	PCS Systemtechnik GmbH
192.168.238.6	0a:00:27:00:00:16	1	60	Unknown vendor
192.168.238.7	08:00:27:9f:36:55	1	60	PCS Systemtechnik GmbH

Figura 2.1: Output del comando netdiscover con l'identificazione degli host attivi.

L'identificazione esatta dell'host bersaglio è avvenuta tramite un processo di esclusione logica. L'output dello scanner ha rivelato la presenza di tre indirizzi IP attivi, oltre a quello della macchina attaccante (192.168.238.4). Avendo mappato l'infrastruttura di base, è stato possibile dedurre che l'indirizzo 192.168.238.2 corrispondeva al gateway logico assegnato di default dall'hypervisor e che l'IP 192.168.238.6 era associato a un servizio interno dell'host fisico.

Di conseguenza, l'unico indirizzo IP rimanente e sconosciuto, ovvero il 192.168.238.7, è stato inequivocabilmente associato alla macchina virtuale **no_name**. A conferma di tale deduzione, l'analisi del campo OUI (Organizationally Unique Identifier) del MAC Address ha restituito come vendor *PCS Systemtechnik GmbH*, la firma digitale standard delle schede di rete emulate da Oracle VirtualBox.

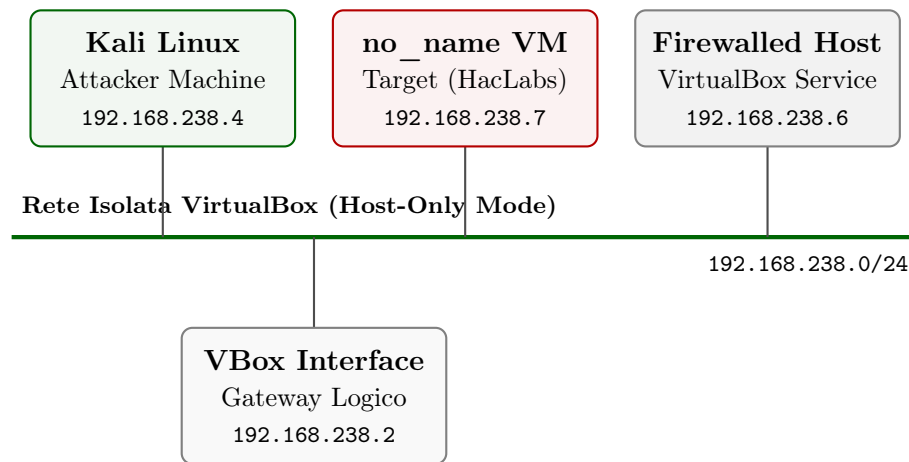


Figura 2.2: Topologia logica dell'ambiente di laboratorio isolato con i nodi attivi rilevati.

2.4 Target Enumeration

Una volta venuti a conoscenza dell'IP della macchina target, è stato possibile passare alla fase successiva di *Enumerazione*, utilizzando strumenti che consentano di ottenere una panoramica a diversi livelli di dettaglio del sistema operativo e dei servizi di rete aperti a tutti i nodi dell'infrastruttura.

2.4.1 OS Fingerprinting (Passivo e Attivo)

Individuato l'indirizzo IP del bersaglio, il passo successivo ha riguardato la profilazione del Sistema Operativo. Al fine di simulare un approccio quanto più stealth e realistico possibile, l'attività è stata suddivisa in due fasi: una prima analisi passiva basata sull'intercettazione del traffico e una successiva validazione attiva.

2.4.2 Fingerprinting Passivo (p0f & cURL)

Il fingerprinting passivo si basa sull'ascolto silente dei pacchetti di rete, senza inviare payload intrusivi al target, analizzando le firme dello stack TCP/IP e i banner applicativi. Per questa fase è stato configurato in ascolto il tool `p0f` sull'interfaccia di rete locale.

Avvio sniffing passivo

```
sudo p0f -i eth1
```

Per generare traffico legittimo da far analizzare allo sniffer, è stata effettuata una semplice richiesta HTTP alla radice del web server del target utilizzando `curl`.


```
(kali㉿kali)-[~/Desktop/no_name_pics]
$ curl http://192.168.238.7/
<h4>Fake Admin Area</h4>
<form action="index.php" method="post">
<input type="text" placeholder="fake query" name="box">
<input type="submit" placeholder="Run" value="submit" name="submitt">
</form>
```

Figura 2.3: Richiesta cURL e risposta HTTP con esposizione del form HTML del target.

L'intercettazione di questo scambio di pacchetti ha permesso a p0f di estrarre preziose informazioni diagnostiche. Sebbene l'analisi iniziale del pacchetto SYN+ACK (firmas TCP) non fosse riuscita a determinare con esattezza il kernel, l'ispezione della risposta HTTP ha rivelato esplicitamente la firma del server web.

```
-[ 192.168.238.4/33976 → 192.168.238.7/80 (mtu) ]-  
| server      = 192.168.238.7/80  
| link        = Ethernet or modem  
| raw_mtu     = 1500  
|_____  
  
-[ 192.168.238.4/33976 → 192.168.238.7/80 (http request) ]-  
| client      = 192.168.238.4/33976  
| app         = ???  
| lang        = none  
| params      = none  
| raw_sig     = 1:Host>User-Agent,Accept=[*/*]:Connection,Accept-Encoding,Accept-Language,Accept-Charset,Keep-Alive:curl/8.19.0  
|_____  
  
-[ 192.168.238.4/33976 → 192.168.238.7/80 (http response) ]-  
| server      = 192.168.238.7/80  
| app         = Apache 2.x  
| lang        = none  
| params      = none  
| raw_sig     = 1:Date,Server,?Vary,?Content-Length,Content-Type:Connection,Keep-Alive,Accept-Ranges:Apache/2.4.29 (Ubuntu)  
|_____
```

Figura 2.4: Output di p0f inerente all'estrazione dei metadati del server web Apache.

Come si evince dall'output (`raw_sig = ... Apache/2.4.29 (Ubuntu)`), il sistema remoto esegue una distribuzione **Ubuntu Linux**. Un'ulteriore deduzione tecnica permette di restringere il campo: la versione 2.4.29 di Apache è storicamente il pacchetto predefinito distribuito sui repository di **Ubuntu 18.04 LTS (Bionic Beaver)**.

2.4.3 Fingerprinting Attivo (Nmap)

Per validare le ipotesi raccolte passivamente, si è proceduto con un OS Fingerprinting attivo tramite Nmap. Questa tecnica, seppur più rumorosa, invia pacchetti TCP/UDP malformati e sonda le risposte specifiche (RFC compliance) per determinare l'esatta architettura del kernel.

```
(kali@kali)-[~]
$ nmap -O 192.168.238.7
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-21 12:28 +0200
Nmap scan report for 192.168.238.7
Host is up (0.00035s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
MAC Address: 08:00:27:9F:36:55 (Oracle VirtualBox virtual NIC)
Device type: general purpose/router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.97 seconds
```

Figura 2.5: Risultato dell'OS Fingerprinting attivo e identificazione euristica del kernel tramite Nmap.

I risultati della scansione attiva hanno confermato le deduzioni precedenti:

- L'host è effettivamente un sistema basato su **Linux**.
- La famiglia del kernel rilevata appartiene alle release **4.X / 5.X** (nello specifico, compatibile con *Linux 4.15 - 5.19*), dato perfettamente coerente con il kernel standard di un'installazione Ubuntu 18.04.
- La scansione ha inoltre confermato l'apertura della **porta 80/tcp** associata al servizio HTTP.

2.4.4 Services Enumeration

Con il fingerprinting siamo riusciti a estrarre non solo informazioni sul sistema operativo (Ubuntu) ma anche a scoprire la presenza di un servizio HTTP sulla porta 80 basato su Apache 2.4.29 e da ciò dedurre una possibile e molto probabile versione del SO che la macchina target ospita (Ubuntu 18.04 LTS). È ora necessario comprendere se il target possiede altri servizi esposti e valutare eventualmente un'attività di ricognizione su tali servizi nell'eventualità che questi presentino vulnerabilità note.

Con l'ausilio di Nmap effettuiamo *port scanning* su `no_name` su tutte e 65535 porte:

```
(kali@kali)-[~]
$ nmap -sV 192.168.238.7 -p- -oX TPC_WM7_SCAN.xml --webxml
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-21 12:33 +0200
Nmap scan report for 192.168.238.7
Host is up (0.000058s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.29 ((Ubuntu))
MAC Address: 08:00:27:9F:36:55 (Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.15 seconds
```

Figura 2.6: Scansione completa del range porta TCP (1-65535) eseguita con Nmap.

Dai risultati si evince che l'unica porta aperta è la 80 e, come volevasi dimostrare, Nmap è riuscito a intuire sia la versione di Apache che il sistema operativo del target. A questo punto è chiaro che il focus del team nella prossima fase del framework sarà l'individuazione di **vulnerabilità web**.

2.4.5 Web Directory Enumeration (Gobuster & Owasp Dirbuster)

Avendo a portata di mano solo ed esclusivamente un web server, risulta fondamentale per il team individuare la directory tree del target, affinché sia possibile avviare l'attività di ispezione delle pagine accessibili pubblicamente e valutare la presenza di elementi interessanti durante la fase di analisi nonché condurre un'attività di directory bruteforcing. Utilizzando `gobuster` e due *wordlist* (`common.txt` e `big.txt`)

```
(kali@kali)~$ gobuster dir -u http://192.168.238.7 -w /usr/share/wordlists/dirb/common.txt -x php,txt,html

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.238.7
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: txt,html,php
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.hta (Status: 403) [Size: 278]
.hta.php (Status: 403) [Size: 278]
.hta.txt (Status: 403) [Size: 278]
.htaccess.php (Status: 403) [Size: 278]
.htaccess (Status: 403) [Size: 278]
.htaccess.txt (Status: 403) [Size: 278]
.htpasswd (Status: 403) [Size: 278]
.htaccess.html (Status: 403) [Size: 278]
.htpasswd.html (Status: 403) [Size: 278]
.htpasswd.php (Status: 403) [Size: 278]
.hta.html (Status: 403) [Size: 278]
.htpasswd.txt (Status: 403) [Size: 278]
admin (Status: 200) [Size: 417]
index.php (Status: 200) [Size: 201]
index.php (Status: 200) [Size: 201]
server-status (Status: 403) [Size: 278]
Progress: 18452 / 18452 (100.00%)

Finished
```

Figura 2.7: Directory Brute-Forcing iniziale tramite Gobuster con l'ausilio della wordlist `common.txt`.

Con `common.txt` siamo riusciti a individuare due pagine accessibili:

- **admin**, molto probabilmente un'area che potrebbe contenere informazioni sensibili sull'amministratore del server o sul server stesso, valutare un'ispezione manuale della pagine e del codice html.
- **index.php**, la pagina di default del server Apache (diventa tale quando si installa l'ambiente php ed ha priorità maggiore rispetto a `index.html`).

Sembra lecito utilizzare una wordlist un po' più grande (`big.txt`) e valutare la presenza di qualche pagina che può essere sfuggita in questa prima fase:

```
(kali@kali)~$ gobuster dir -u http://192.168.238.7 -w /usr/share/wordlists/dirb/big.txt -x php,txt,html

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.238.7
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/big.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: php,txt,html
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

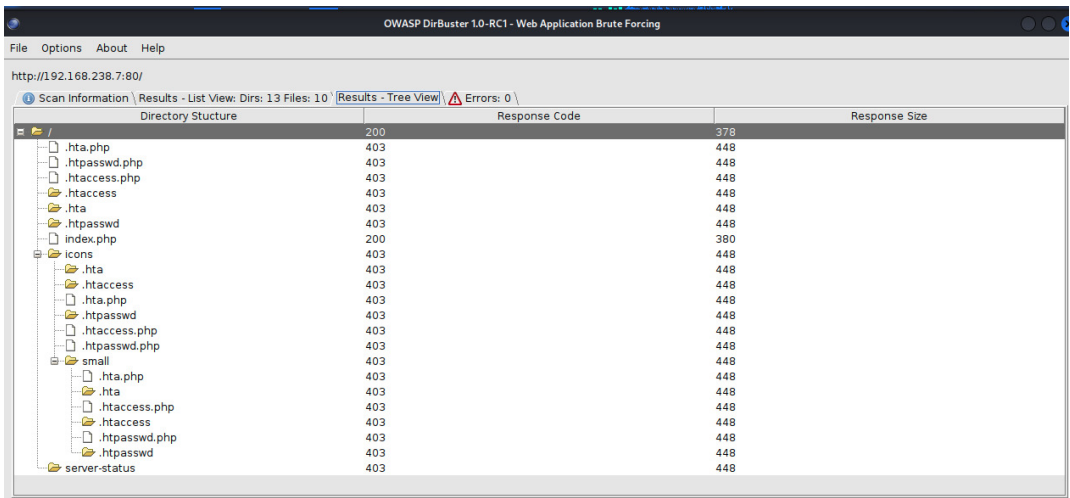
.htaccess.php (Status: 403) [Size: 278]
.htpasswd.txt (Status: 403) [Size: 278]
.htaccess (Status: 403) [Size: 278]
.htpasswd (Status: 403) [Size: 278]
.htaccess.txt (Status: 403) [Size: 278]
.htaccess.html (Status: 403) [Size: 278]
.htpasswd.php (Status: 403) [Size: 278]
.htpasswd.html (Status: 403) [Size: 278]
admin (Status: 200) [Size: 417]
index.php (Status: 200) [Size: 201]
server-status (Status: 403) [Size: 278]
superadmin.php (Status: 200) [Size: 152]
Progress: 81876 / 81876 (100.00%)

Finished
```

Figura 2.8: Directory Brute-Forcing esteso con Gobuster tramite la wordlist `big.txt`.

Questo secondo scan ha individuato una nuova pagina accessibile (`/superadmin.php`), che molto probabilmente rappresenta il fulcro dell'intera opera di Penetration Testing in quanto possibile interfaccia di

amministrazione. Ma andiamo con calma e passiamo all’individuazione tramite **OWASP - Dirbuster** della gerarchia dei file del server ottenendo il seguente risultato:



Directory Structure	Response Code	Response Size
/	200	378
.hta.php	403	448
.htpasswd.php	403	448
.htaccess.php	403	448
.htaccess	403	448
.hta	403	448
.htpasswd	403	448
index.php	200	380
icons	403	448
.hta	403	448
.htaccess	403	448
.hta.php	403	448
.htpasswd	403	448
.htaccess.php	403	448
.htpasswd.php	403	448
small	403	448
.hta.php	403	448
.hta	403	448
.htaccess.php	403	448
.htaccess	403	448
.htpasswd.php	403	448
.htpasswd	403	448
server-status	403	448

Figura 2.9: Schermata dei risultati strutturali in modalità Tree View estratti da OWASP DirBuster.

Possiamo schematizzare l’output di Dirbuster con l’ausilio del seguente grafico ad albero e integrarlo con quello di gobuster per ottenere una panoramica del directory tree del web server accurata:

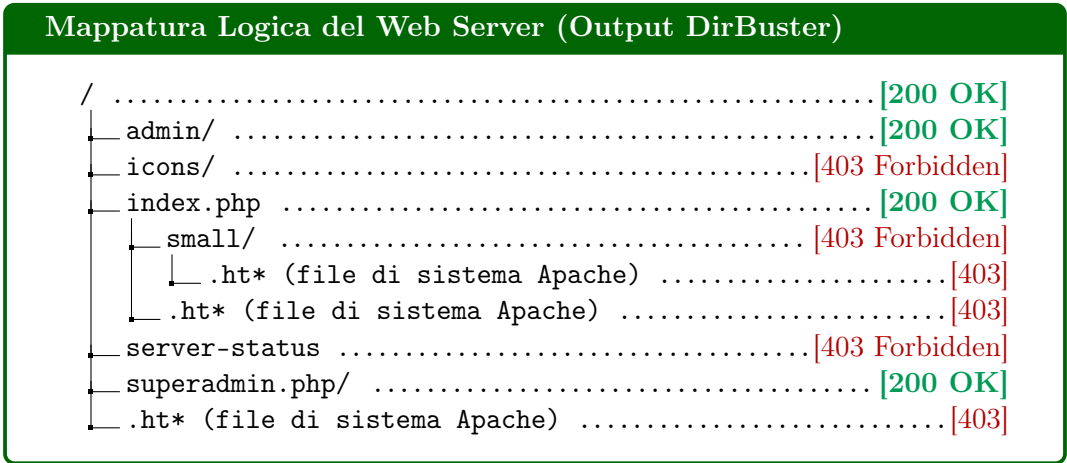


Figura 2.10: Grafico strutturale definitivo della gerarchia dei file esposti dal server web del target.

L’analisi dell’albero evidenzia la presenza di tre pagine web, rappresentando di fatto la superficie d’attacco registrata e analizzabile dal team. Tutte queste pagine hanno restituito un codice di stato **200 OK**.

2.5 Vulnerability Assessment

2.6 Target Exploitation

2.7 Privilege Escalation

2.8 Maintaining Access

Capitolo 3

Final Consideration

Capitolo 4

References